

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
CAMPUS LIBERDADE — BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

ENTREGA 2

Transformações Lineares em Imagens Digitais

Álgebra Linear, Vetores e Geometria Analítica

Grupo 8 · Radonix

Gabriel Carvalho Mota | Guilherme de Lima Siqueira
Rodrigo Luiz Menezes dos Reis | Vitoria Leticia Maciel da Silva

São Paulo, 2026

Sumário

1 Visão Geral do Projeto	3
2 Bases Visuais e Insumos	3
3 Lógica de Desenvolvimento	3
4 Ferramentas e Funções Utilizadas	5
5 Transformações Lineares Implementadas	6
5.1 Escalonamento (Scaling)	6
5.2 Cisalhamento (Shear)	7
5.3 Rotação	8
5.4 Composição de Transformações	9
5.5 Colapso Dimensional (Projeção Singular)	10
6 Quadro Comparativo das Transformações	11
7 Arquivos Gerados	11
8 Conclusão	12

1 Visão Geral do Projeto

Este notebook Python implementa cinco transformações lineares aplicadas a uma imagem digital, gerando para cada transformação um arquivo Excel colorido — mantendo o mesmo padrão visual do Entregável 1 (células coloridas com valor numérico oculto). A abordagem une conceitos fundamentais de Álgebra Linear com processamento digital de imagens, tornando visível e interativo o efeito de cada transformação sobre uma matriz de pixels.

Atributo	Valor
Disciplina	Álgebra Linear, Vetores e Geometria Analítica
Grupo	Grupo 8 · Radonix
Linguagem	Python 3 (Google Colab)
Saída	6 arquivos .xlsx (original + 5 transformações)
Resolução	200 × 200 pixels por imagem

2 Bases Visuais e Insumos

O notebook aceita qualquer imagem PNG ou JPG enviada pelo usuário via upload interativo do Google Colab. Ao carregar a imagem, dois fluxos paralelos são criados:

- Fluxo RGB — preserva as três componentes de cor (Vermelho, Verde, Azul) para colorir as células do Excel.
- Fluxo Grayscale (L) — converte a imagem para tons de cinza e usa o valor de luminância (0–255) como ID numérico oculto em cada célula.

Ambos os fluxos são redimensionados para 200 × 200 pixels usando o filtro LANCZOS, que oferece qualidade superior ao NEAREST utilizado na Entrega 1, preservando suavidade nas bordas.

3 Lógica de Desenvolvimento

3.1 Decomposição Espacial — Pixels para Matriz

A imagem carregada é convertida em dois arrays NumPy:

```
mat_cores = np.array(img_rgb) # shape (200, 200, 3) — valores R, G, B por pixel
mat_numeros = np.array(img_gray) # shape (200, 200) — valor 0-255 por pixel
```

Cada pixel ocupa exatamente uma célula Excel de 2,5 largura × 15 pt altura, reproduzindo fielmente a grade visual do Entregável 1.

3.2 Mapeamento Inverso para Transformações

A função central `aplicar_transformacao(M, rgb, gray)` realiza a transformação usando mapeamento inverso, técnica padrão em processamento de imagens para evitar buracos (pixels sem atribuição):

Passo a passo:

Passo 1 — Grade de coordenadas centradas: o sistema de coordenadas é reposicionado para o centro da imagem ($cx = cy = 100.0$), de modo que as transformações girem/escalem em torno do centro e não do canto superior esquerdo.

```
cx, cy = W / 2.0, H / 2.0
cols_grid, rows_grid = np.meshgrid(np.arange(W), np.arange(H))
coords_dest = np.stack([
    cols_grid.ravel() - cx,    # coordenada x centralizada
    rows_grid.ravel() - cy    # coordenada y centralizada
]) # shape (2, 40000) - todos os 40.000 pixels vetorizados
```

Passo 2 — Inversão da matriz: para cada pixel de destino (d), é calculado o pixel de origem (s) aplicando a inversa de M :

$$s = M^{-1} \cdot d$$

```
M_inv = np.linalg.inv(M)          # calcula a inversa de M
src    = M_inv @ coords_dest      # aplica: s = M_inv · d (vetorizado)
src_row = src[1] + cy             # reconverte para índice de linha
src_col = src[0] + cx             # reconverte para índice de coluna
```

Passo 3 — Interpolação bilinear: como as coordenadas de origem podem ser fracionárias (ex.: pixel 73,4), `map_coordinates` interpola entre os pixels vizinhos com `order=1` (bilinear), resultando em transições suaves:

```
for c in range(3): # para cada canal R, G, B
    rgb_t[..., c] = map_coordinates(
        rgb[..., c].astype(float),
        [src_row.reshape(H, W), src_col.reshape(H, W)],
        order=1, # interpolação bilinear
        mode='constant', # pixels fora da imagem recebem 0 (preto)
        cval=0
    ).astype(np.uint8)
```

3.3 Processamento de Cor — RGB para Hexadecimal

O arquivo Excel precisa de códigos hexadecimais (`#RRGGBB`) para colorir as células. A conversão é feita na função `salvar_excel`:

```
rgb = mat_rgb[r, c]
hex_color = '#{0:02x}{1:02x}{2:02x}'.format(int(rgb[0]), int(rgb[1]), int(rgb[2]))
```

Um cache de formatos (dict Python) garante que cores iguais reutilizem o mesmo objeto de formato, evitando o limite de 64.000 formatos do `XlsxWriter` e reduzindo o tamanho do arquivo `.xlsx`.

3.4 Camada de Dados Ocultos

Cada célula armazena simultaneamente a cor (bg_color) e um número inteiro (valor_id = nível de cinza do pixel). A formatação ';;;' oculta o número visualmente, mas ele pode ser consultado na barra de fórmulas do Excel — exatamente como no Entregável 1.

```
formatos_cache[hex_color] = workbook.add_format({
    'bg_color': hex_color, # cor de fundo = cor do pixel
    'num_format': ';;;' # oculta o número visualmente
})
worksheet.write(r, c, valor_id, formatos_cache[hex_color])
```

4 Ferramentas e Funções Utilizadas

A entrega foi desenvolvida em Python, combinando bibliotecas especializadas:

4.1 Pillow (PIL)

- `Image.open()` + `io.BytesIO()` — leitura da imagem a partir de bytes do upload do Colab.
- `.convert('RGB')` — garante três canais de cor (elimina canal alfa se presente).
- `.convert('L')` — converte para escala de cinza (luminância ponderada: $0.299R + 0.587G + 0.114B$).
- `.resize((200, 200), Image.LANCZOS)` — redimensionamento de alta qualidade com filtro anti-aliasing.

4.2 NumPy

- `np.array()` — converte imagens PIL em arrays multidimensionais de alto desempenho.
- `np.meshgrid()` + `np.stack()` — cria a grade de coordenadas vetorizada (40.000 pontos) em uma única operação.
- `np.linalg.det()` — calcula o determinante da matriz de transformação.
- `np.linalg.inv()` — calcula a matriz inversa para o mapeamento inverso.
- `np.radians()` — converte ângulos de graus para radianos.
- `np.zeros_like()` — aloca matrizes de saída zeradas com mesma forma/tipo que a entrada.

4.3 SciPy — map_coordinates

- `map_coordinates(input, coords, order=1, mode='constant', cval=0)` — realiza interpolação bilinear nas coordenadas de origem calculadas pelo mapeamento inverso. É a função que torna as transformações suaves, sem serrilhado.

4.4 XlsxWriter

- `Workbook / Worksheet` — cria e gerencia o arquivo .xlsx.
- `set_column(0, 199, 2.5)` — define largura de 2,5 para todas as 200 colunas.
- `set_row(r, 15)` — define altura de 15 pt para cada linha.

- `add_format({'bg_color': ..., 'num_format': ';;;'})` — cria formatos reutilizáveis com cor e número oculto.
- `worksheet.write(r, c, valor_id, formato)` — escreve o dado em cada célula.

4.5 Pandas

Utilizado implicitamente na Entrega 1 para geração do arquivo CSV de backup. Na Entrega 2 permanece importado para eventual exportação de dados tabulares das matrizes transformadas.

4.6 Matplotlib

- `plt.imshow()` — renderiza a imagem original e a transformada lado a lado para comparação visual.
- `plt.subplots(1, 2)` — organiza dois painéis de comparação em cada seção.

5 Transformações Lineares Implementadas

Todas as transformações são aplicadas sobre coordenadas centradas no pixel (100, 100), garantindo que escala, rotação e cisalhamento atuem a partir do centro da imagem e não do canto superior esquerdo.

5.1 Transformação 1 — Escalonamento (Scaling)

$$M_{\text{escala}} = \begin{vmatrix} s_x & 0 \\ 0 & s_y \end{vmatrix} \quad \text{com } s_x = 1.5, \quad s_y = 0.75$$

Conceito matemático:

- Cada coordenada x é multiplicada por s_x e cada coordenada y por s_y .
- $s_x > 1$ expande horizontalmente; $s_x < 1$ contrai. Mesma lógica para s_y .
- Valor negativo de s_x ou s_y inverte o eixo correspondente (espelhamento).

Cálculo do determinante:

$$\det(M_{\text{escala}}) = s_x \times s_y = 1.5 \times 0.75 = 1.125$$

Como $\det \neq 0$, a matriz é inversível e o mapeamento inverso pode ser aplicado.

Cálculo da inversa (mapeamento inverso):

$$M_{\text{escala}}^{-1} = \begin{vmatrix} 1/s_x & 0 \\ 0 & 1/s_y \end{vmatrix} = \begin{vmatrix} 0.667 & 0 \\ 0 & 1.333 \end{vmatrix}$$

Implementação:

```
sx, sy = 1.5, 0.75
M_escala = np.array([[sx, 0], [0, sy]], dtype=float)
print(f'det(M) = {np.linalg.det(M_escala):.4f}') # 1.1250
rgb_esc, gray_esc = aplicar_transformacao(M_escala, mat_cores, mat_numeros)
salvar_excel(rgb_esc, gray_esc, 'T1_escalonamento.xlsx')
```

Efeito visual: a imagem aparece comprimida verticalmente ($s_y = 0.75$) e expandida horizontalmente ($s_x = 1.5$). Regiões que saem do canvas ficam pretas ($cval=0$).

5.2 Transformação 2 — Cisalhamento (Shear)

$$M_{\text{cis}} = \begin{vmatrix} 1 & k_x \\ k_y & 1 \end{vmatrix} \quad \text{com } k_x = 0.4, \quad k_y = 0.0$$

Conceito matemático:

- $k_x \neq 0$ inclina cada coluna de pixels proporcionalmente à sua posição em y (cisalhamento horizontal).
- $k_y \neq 0$ inclina cada linha proporcionalmente a x (cisalhamento vertical).
- Geometricamente: quadrados se transformam em paralelogramos.

Cálculo do determinante:

$$\det(M_{\text{cis}}) = 1 \times 1 - k_x \times k_y = 1 - 0.4 \times 0 = 1.0$$

Determinante = 1 preserva a área total da imagem.

Cálculo da inversa (fórmula geral 2×2 : $M^{-1} = (1/\det) \cdot [[d, -b], [-c, a]]$):

$$M_{\text{cis}}^{-1} = (1/\det) \times \begin{vmatrix} 1 & -k_x \\ -k_y & 1 \end{vmatrix} = \begin{vmatrix} 1 & -0.4 \\ 0 & 1 \end{vmatrix} \quad (\det = 1)$$

Implementação:

```
kx, ky = 0.4, 0.0
M_cis = np.array([[1, kx], [ky, 1]], dtype=float)
print(f'det(M) = {np.linalg.det(M_cis):.4f}') # 1.0000
rgb_cis, gray_cis = aplicar_transformacao(M_cis, mat_cores, mat_numeros)
salvar_excel(rgb_cis, gray_cis, 'T2_cisalhamento.xlsx')
```

Efeito visual: a imagem é "inclinada" para a direita, como um paralelogramo — linhas horizontais permanecem horizontais, mas deslocadas proporcionalmente à altura.

5.3 Transformação 3 — Rotação

$$M_{\text{rot}}(\theta) = \begin{vmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{vmatrix} \quad \text{com } \theta = 45^\circ$$

Substituindo $\theta = 45^\circ = \pi/4$:

$$M_{\text{rot}} = \begin{vmatrix} \cos(45^\circ) & -\sin(45^\circ) \\ \sin(45^\circ) & \cos(45^\circ) \end{vmatrix} \approx \begin{vmatrix} 0.7071 & -0.7071 \\ 0.7071 & 0.7071 \end{vmatrix}$$

Propriedades matemáticas:

- Transformação ortogonal: preserva todas as distâncias e ângulos (isometria).
- $\det(M_{\text{rot}}) = \cos^2(\theta) + \sin^2(\theta) = 1$ sempre — nunca colapsa o espaço.
- A inversa é igual à transposta: $M_{\text{rot}}^{-1} = M_{\text{rot}}^T$ (propriedade de matrizes ortogonais).

Cálculo da inversa (no caso da rotação, é a rotação de $-\theta$):

$$M_{\text{rot}}^{-1} = M_{\text{rot}}^T = \begin{vmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{vmatrix} \approx \begin{vmatrix} 0.7071 & 0.7071 \\ -0.7071 & 0.7071 \end{vmatrix}$$

Implementação:

```
angulo_graus = 45
theta = np.radians(angulo_graus)          # 45° → π/4 rad
M_rot = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta),  np.cos(theta)]
])
print(f'det(M) = {np.linalg.det(M_rot):.4f}') # 1.0000
rgb_rot, gray_rot = aplicar_transformacao(M_rot, mat_cores, mat_numeros)
salvar_excel(rgb_rot, gray_rot, 'T3_rotacao.xlsx')
```

Efeito visual: a imagem gira 45° no sentido anti-horário em torno do centro. As bordas ficam pretas pois os cantos da imagem girada extrapolam o canvas 200×200.

5.4 Transformação 4 — Composição de Transformações

$$M_{\text{comp}} = M_{\text{rot}} \times M_{\text{escala}} \times M_{\text{cis}}$$

Conceito matemático:

- A multiplicação de matrizes compõe transformações: a transformação mais à direita é aplicada primeiro.
- Ordem de aplicação: (1) Cisalhamento → (2) Escalonamento → (3) Rotação.
- A composição NÃO é comutativa: mudar a ordem altera o resultado.

Determinante da composição (propriedade multiplicativa):

$$\begin{aligned} \det(M_{\text{comp}}) &= \det(M_{\text{rot}}) \times \det(M_{\text{escala}}) \times \det(M_{\text{cis}}) \\ &= 1.000 \times 1.125 \times 1.000 = 1.125 \end{aligned}$$

Inversa da composição — pela propriedade $(AB)^{-1} = B^{-1}A^{-1}$, a ordem se inverte:

$$M_{\text{comp}}^{-1} = M_{\text{cis}}^{-1} \times M_{\text{escala}}^{-1} \times M_{\text{rot}}^{-1}$$

Implementação:

```
M_comp = M_rot @ M_escala @ M_cis      # @ = multiplicação matricial
print(np.round(M_comp, 4))
print(f'det(M_comp) = {np.linalg.det(M_comp):.4f}') # 1.1250
rgb_comp, gray_comp = aplicar_transformacao(M_comp, mat_cores, mat_numeros)
salvar_excel(rgb_comp, gray_comp, 'T4_composicao.xlsx')
```

Efeito visual: a imagem é simultaneamente inclinada, comprimida/expandida e rotacionada, resultando em uma deformação complexa que combina os efeitos individuais.

5.5 Transformação 5 — Colapso Dimensional (Projeção Singular)

$$M_{\text{proj}} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad \det(M_{\text{proj}}) = 0 \rightarrow \text{SINGULAR}$$

Conceito matemático:

- Quando $\det = 0$, a matriz é singular e NÃO tem inversa — o mapeamento inverso é impossível.
- Esta matriz projeta todos os pontos sobre o eixo X ($y \leftarrow 0$): toda coordenada Y é zerada.
- O espaço 2D colapsa para 1D — toda a imagem é "achatada" em uma única linha horizontal central.
- É o único caso que usa mapeamento direto (origem $\rightarrow$ destino) no código.

Por que mapeamento direto?

Como não existe M_{proj}^{-1} , o algoritmo percorre os pixels de origem e calcula onde eles vão para o destino — ao invés da lógica inversa usada nas demais transformações.

```
M_proj = np.array([[1, 0], [0, 0]], dtype=float)
# det = 0 → mapeamento direto (origem → destino)
rgb_proj = np.zeros((H, W, 3), dtype=np.uint8)
gray_proj = np.zeros((H, W), dtype=np.uint8)

for r in range(H):
    for c in range(W):
        xc, yc = c - cx, r - cy          # centraliza
        ponto = M_proj @ np.array([xc, yc]) # aplica projeção
        dc = int(np.round(ponto[0] + cx))    # coluna de destino
        dr = int(np.round(ponto[1] + cy))    # linha de destino (sempre 100)
        if 0 <= dr < H and 0 <= dc < W:
            rgb_proj[dr, dc] = mat_cores[r, c] # sobrescreve para linha 100
            gray_proj[dr, dc] = mat_numeros[r, c]
```

Efeito visual: toda a imagem colapsa para a linha central (linha 100 de 200), criando uma faixa horizontal colorida. O restante da imagem fica preto, demonstrando a perda irreversível de informação quando o determinante é zero.

6 Quadro Comparativo das Transformações

Transformação	Matriz M	Determinante	Propriedade
T0 – Original	I (identidade)	1.0000	Referência
T1 – Escala	$\begin{bmatrix} 1.5, & 0 \\ 0, & 0.75 \end{bmatrix}$	1.1250	Inversível
T2 – Cisalham.	$\begin{bmatrix} 1, & 0.4 \\ 0, & 1 \end{bmatrix}$	1.0000	Área preservada
T3 – Rotação 45°	$\begin{bmatrix} 0.707, & -0.707 \\ 0.707, & 0.707 \end{bmatrix}$	1.0000	Ortogonal
T4 – Composição	M_rot · M_esc · M_cis	1.1250	Produto det's
T5 – Projeção	$\begin{bmatrix} 1, & 0 \\ 0, & 0 \end{bmatrix}$	0.0000	SINGULAR – sem inv.

Observação: o determinante da composição (T4) é igual ao produto dos determinantes individuais — propriedade fundamental do produto matricial: $\det(AB) = \det(A) \cdot \det(B)$.

7 Arquivos Gerados

Ao executar todas as células do notebook, seis arquivos Excel são gerados e baixados automaticamente:

Arquivo	Transformação	Determinante
T0_original.xlsx	Imagem original (referência)	det = 1
T1_escalonamento.xlsx	Escala (sx=1.5, sy=0.75)	det = 1.125
T2_cisalhamento.xlsx	Cisalhamento (kx=0.4, ky=0)	det = 1.0
T3_rotacao.xlsx	Rotação 45°	det = 1.0
T4_composicao.xlsx	Composição (rot · esc · cis)	det = 1.125
T5_colapso_dimensional.xlsx	Projeção (colapso para eixo X)	det = 0 → singular

8 Conclusão

A Entrega 2 expande a abordagem da Entrega 1 ao aplicar cinco classes distintas de transformações lineares sobre a matriz de pixels de uma imagem, tornando tangível o impacto de cada operação algébrica sobre dados visuais.

Os principais aprendizados demonstrados pelo projeto são:

- Toda transformação linear 2D pode ser representada por uma matriz 2×2 , e seu efeito sobre a imagem é completamente determinado pelos coeficientes dessa matriz.
- O determinante é o indicador crítico de invertibilidade: $\det \neq 0$ permite mapeamento inverso (sem perda de informação); $\det = 0$ torna a transformação irreversível.
- O mapeamento inverso com interpolação bilinear (map_coordinates) é a técnica correta para transformações em imagens digitais, evitando buracos e serrilhado.
- A composição de matrizes (produto matricial) permite combinar múltiplas transformações em uma única operação, com o determinante composto igual ao produto dos determinantes individuais.
- O colapso dimensional (projeção singular) demonstra concretamente o conceito de perda de posto (rank deficiency) e sua consequência prática: destruição irreversível de informação.

O resultado final é um conjunto de seis planilhas Excel que funcionam como uma interface visual interativa, permitindo inspecionar o ID numérico (intensidade de cinza) de qualquer pixel transformado diretamente na barra de fórmulas.